



M2ITIC/IFOAD

Mise à niveau JAVA

Fiche TD/TP 3 : Programmation Orientée Objet (POO)

OBJECTIFS :

- Ecrire des programmes en définissant des classes.

Exercice 1 : Palindrome

Un palindrome est un mot qui peut se lire de droite à gauche ou de gauche à droite. Par exemple le mot "elle" est un palindrome. Écrire une classe Palindrome qui contient :

- 1* une méthode **public static int cmpCharAt(String s, int i, int j);** qui compare le i-ème et le j-ème caractère de s et retourne 1 s'ils sont égaux et 0 s'ils sont différents.
- 2* une méthode **public static int palin(String s);** contenant une boucle permettant de déterminer si un mot passé en paramètre est un palindrome. Les réponses possibles sont les suivantes : 1 si la chaîne est un palindrome et 0 sinon.
- 3* une méthode **public static void main(String[] args);** de test de vos méthodes.

Exercice 2 : Point

Créer une classe permettant de manipuler un point d'un axe, repéré par une abscisse (de type *int*). On devra pouvoir effectuer des changements d'origine, en conservant en permanence l'abscisse d'une origine courante (initialement 0). On prévoira simplement les méthodes suivantes :

- constructeur, recevant en argument l'abscisse "absolue" du point (c'est-à-dire repérée par rapport au point d'origine 0 et non par rapport à l'origine courante),
- *affiche* qui imprime à la fois l'abscisse de l'origine courante et l'abscisse du point par rapport à cette origine,

- *setOrigine* qui permet de définir une nouvelle abscisse pour l'origine (exprimée de façon absolue et non par rapport à l'origine courante),
- *getOrigine* qui permet de connaître l'abscisse de l'origine courante.

Ecrire un petit programme de test fournissant les résultats suivants :

Point a - abscisse = 3
relative a une origine d'abscisse 0
Point b - abscisse = 12
relative a une origine d'abscisse 0
On place l'origine en 3

Point a - abscisse = 0
relative a une origine d'abscisse 3
Point b - abscisse = 9
relative a une origine d'abscisse 3

Exercice 3 : Simulation d'une carte téléphonique

Une carte téléphonique de 50 unités peut être représentée sous forme d'un objet avec les messages suivants :

- **debiterUnites (debit)** : débite une carte d'un nombre variable d'unités **debit** ;
- **getUnites** : renvoie le nombre d'unités restantes d'une carte.

Ecrire une classe Java qui simule une carte téléphonique.

Exercice 4 : Ensemble d'entiers ordonnés

Un ensemble d'entiers ordonné est caractérisé par la taille maximale de l'ensemble, le tableau contenant les éléments, l'indice courant dans le tableau, les primitives d'insertion, de suppression, de test d'appartenance, de successeur, de prédécesseur, d'union, d'intersection.

1. Définir une classe Java **EntOrd** qui implémente le concept d' « *ensemble d'entiers ordonnés* ».

La classe doit au moins contenir les méthodes suivantes :

- public void insertion(int x)
- public void suppression(int x)
- public boolean appartient(int x)
- public int succ(int x)
- public int pred(int x)
- public EntOrd union(EntOrd e1)
- public EntOrd intersection(EntOrd e1)

2. Ecrire une classe **CreerEntOrd** qui permet de créer une instance de la classe **EntOrd** à partir d'entiers saisis comme arguments de la ligne de commande.

Exercice 5 : Représentation et Manipulation de Polynômes

Un polynôme (par exemple $2-3x^2+x^4$) peut être représenté par un tableau comme suit :

0	1	2	3	4
2	0	-3	0	1

1) En adoptant cette représentation, définir une classe Java nommée **Polynome** permettant de réaliser les opérations suivantes :

- construction d'un polynôme à partir d'une notation vectorielle (par exemple 2, 0, -3, 0, 1 pour le polynôme précédant)
- addition de deux polynômes
- affichage d'un polynôme
- dérivée d'un polynôme
- calcul de la valeur d'un polynôme pour un réel donné.

2) Ecrivez un programme principal :

- saisissant un polynôme P
- calculant et affichant $P + 2-3x^2+x^4$
- calculant et affichant la dérivée de P en 2

3) Construire une classe **Polynome2** adaptée aux polynômes de degré 2 à partir de la classe **Polynome**. Cette classe doit posséder une méthode nommée **ResoudreEquation** cherchant les solutions de l'équation du second degré associée au polynôme.

Exercice 6 : Matériel informatique

On se propose de développer une application pour un magasin spécialisé dans la vente de matériels informatiques : ordinateurs (PCs et portables) et périphériques (imprimantes, scanners).

Tout produit du magasin est caractérisé par une référence, une marque, une date de fabrication et un prix.

Un ordinateur est considéré comme un produit caractérisé par son processeur (ex : Intel Dual Core T2370 à 1,73 GHz), la taille de sa RAM et du disque dur exprimées en Giga-octets et le système d'exploitation avec lequel il est livré. Pour les ordinateurs de bureau (PC), on a l'information sur la carte graphique alors pour les ordinateurs portables, on a les caractéristiques concernant l'écran (type et taille).

Les imprimantes sont caractérisées par leur type (laser, jet-d'encre, ...), leur résolution et leur vitesse d'impression. Les scanners sont caractérisés par leur résolution.

1. Ecrire une classe **Produit_Informatique** permettant de représenter les équipements vendus par le magasin. La classe devra comporter au moins un constructeur et des méthodes de type *get* et *set*.
2. Ecrire des classes qui héritent de la classe **Produit_Informatique** et permettent de représenter les ordinateurs et les périphériques. Chaque classe devra comporter au moins un constructeur et des méthodes de type *get* et *set*.
3. Ecrire une classe **Test** qui contiendra une méthode *main* permettant de créer et de manipuler des instances des classes créées précédemment.

Exercice 7 : Racine carrée

1. Ecrire une classe **Racine** servant à calculer la racine carrée d'un nombre réel de type **double**.

La classe **Racine** doit contenir un attribut privé nommé **precision** dont la valeur est initialisée lors de l'instanciation de la classe et modifiable via une méthode *set*.

Il faut que la méthode **calculerRacine** qui permet de calculer la racine lance une exception dans le cas où le nombre double qui lui est passé en paramètre est négatif. Pour cela, on définira une classe étendant la classe **Exception**, correspondant à la classe de l'exception qui sera levée dans le cas d'un nombre négatif. Dans ce cas, on lancera une exception de la classe définie depuis l'intérieur de la méthode **calculerRacine** de la classe **Racine**.

Pour le calcul de la racine carrée du nombre, on utilisera une méthode dichotomique. On initialisera l'intervalle sur lequel sera recherchée la solution avec l'intervalle $[0, \text{nombre}]$ et la première solution dont le carré sera comparé au nombre sera le milieu de cet intervalle. On réitérera jusqu'à ce que la différence entre les deux bornes de l'intervalle soit inférieure ou égale à la valeur de la précision.

2. Ecrire une classe **CalculRacine** qui permet d'utiliser la classe **Racine**.

Le nombre à traiter sera envoyé comme argument sur la ligne de commande.

La valeur de la précision sera saisie en utilisant la classe **java.util.Scanner** ou la classe **javax.swing.JOptionPane**.

Le résultat sera affiché sous la forme : « *La racine de carrée de X est Y avec une précision de P* »

Exercice 8 : Utilisateurs

On veut développer une classe Java, nommée *User*, permettant de représenter des utilisateurs d'un système. Chaque utilisateur est caractérisé par un *login*, un *mot de passe* (chaîne de 11 caractères exactement), un *nom* et un *prénom*.

Ecrire la classe *User* en fournissant des méthodes *get* et *set* pour les attributs. Le constructeur doit permettre de créer un utilisateur à partir des informations concernant son login, son nom

et son prénom. Un mot de passe initial ayant la même valeur que le login doit être automatiquement fourni à l'utilisateur en attendant qu'il puisse le modifier au moyen de la méthode permettant de modifier le mot de passe. Cette méthode doit générer une exception de type *MotDePasseInvalideException* (à définir bien sûr) si le mot de passe fourni par l'utilisateur ne respecte pas la règle de définition des mots de passe.

Ecrire une classe *UserManager* qui crée d'abord un objet utilisateur dont les informations sont saisies au moyen du composant Swing *JOptionPane* avant de lui faire changer son mot de passe.

Dire pourquoi les méthodes *get* et *set* doivent être publiques.

Exercice 9 : Moyenne de nombres

Ecrire un programme **Java** qui à partir d'un tableau d'entiers représentés en chaînes de caractères (*String[]* liste) calcule la moyenne de ces nombres. Le programme doit traiter le cas où une chaîne de caractères ne correspond pas à un entier en provoquant une exception. Le programme doit aussi traiter le cas où l'entier n'est pas compris dans un intervalle donné [*min*, *max*].

Exercice 10 : Déclenchement d'une exception

Réaliser une classe *EntNat* permettant de manipuler des entiers naturels (positifs ou nuls). Pour l'instant, cette classe disposera simplement :

- d'un constructeur à un argument de type *int* qui générera une exception de type *ErrConst* (type classe à définir) lorsque la valeur reçue ne conviendra pas,
- d'une méthode *getN* fournissant sous forme d'un *int*, la valeur encapsulée dans un objet de type *EntNat*.

Écrire un petit programme d'utilisation qui traite l'exception *ErrConst* en affichant un message et en interrompant l'exécution.

Adapter la classe *EntNat* de l'exercice et le programme d'utilisation de manière à disposer dans le gestionnaire d'exception du type *ErrConst* de la valeur fournie à tort au constructeur.

Exercice 11 : Entier Naturel

Réaliser une classe permettant de manipuler des entiers naturels (positifs ou nuls) et disposant :

- d'un constructeur à un argument de type *int* ; il générera une exception *ErrConst* si la valeur de son argument est négative ;
- de méthodes statiques de somme, de différence et de produit de deux naturels ; elles généreront respectivement des exceptions *ErrSom*, *ErrDiff* et *ErrProd* lorsque le résultat ne sera pas représentable ; la limite des valeurs des naturels sera fixée à la plus grande valeur du type *int* ;

- une méthode d'accès *getN* fournissant sous forme d'un *int* la valeur de l'entier naturel. On s'arrangera pour que toutes les classes exception dérivent d'une classe *ErrNat* et pour qu'elles permettent à un éventuel gestionnaire de récupérer les valeurs ayant provoqué l'exception.

Écrire deux exemples d'utilisation de la classe :

- l'un se contentant d'intercepter sans discernement les exceptions de type dérivé de *ErrNat*,
- l'autre qui explicite la nature de l'exception en affichant les informations disponibles. Les deux exemples pourront figurer dans deux blocs *try* d'un même programme.